

Détection de mouvement

Implémentation dans le moteur de jeu Godot



Sommaire

I. MediaPipe

II. Modules Camera dans Godot

III. Extensions MediaPipe dans Godot



MediaPipe



MediaPipe Solutions

- MediaPipe Solutions est un service proposé par **Google** fournit une suite de bibliothèques et d'outils qui permettent d'appliquer rapidement **des techniques d'intelligence artificielle (IA) et de machine learning (ML)** dans des applications.
- Parmi ces solutions, on retrouve notamment la détection de gestes, la détection des points de repère sur les mains, le visage, le corps, ...

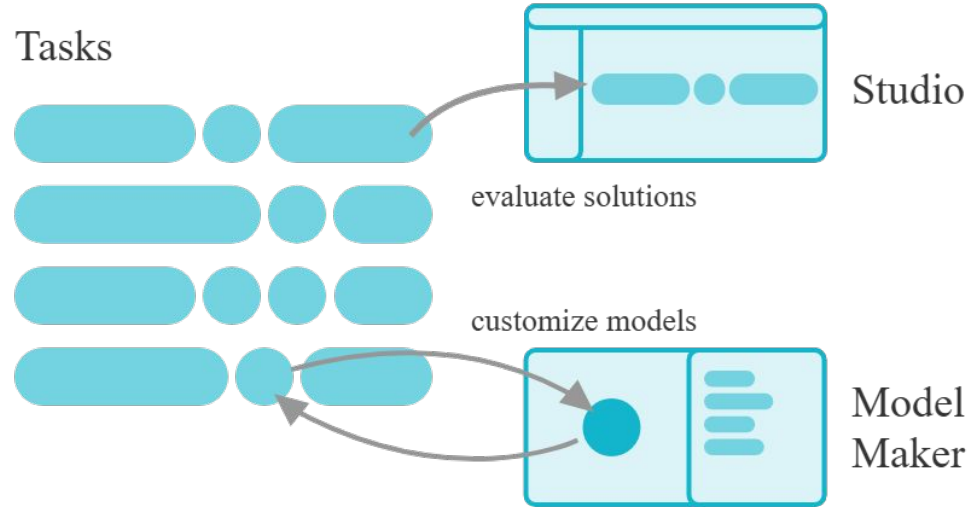
Description des solutions

MediaPipe Tasks : API et bibliothèques multiplates-formes pour déployer des solutions.

Modèles MediaPipe : modèles pré-entraînés et prêts à l'emploi à utiliser avec chaque solution.

MediaPipe Model Maker : personnalisez les modèles pour les solutions avec vos données.

MediaPipe Studio : visualisez, évaluez et comparez les solutions dans votre navigateur.

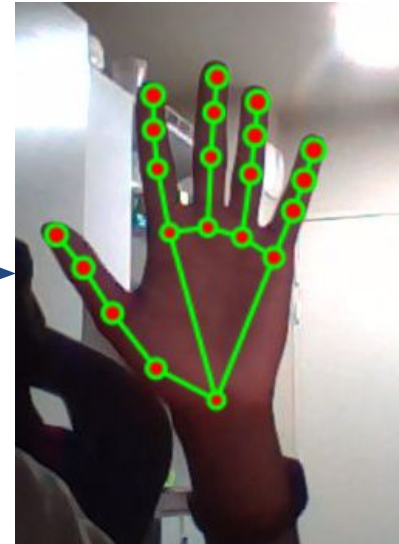


MediaPipe Studio

Exemple d'utilisation de MediaPipe Studio



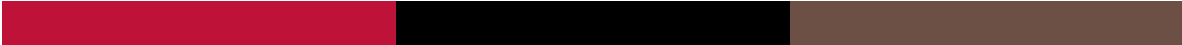
Traitement





MediaPipe Tasks

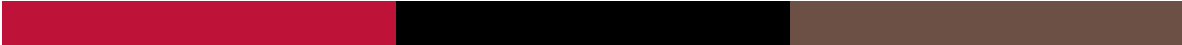
- Plusieurs API sont mises en place pour employer MediaPipe dans d'autres applications (Python, Web, etc...)
- Le fonctionnement de ces API est donnée par **les fichiers *tasks***



Godot : Module Camera

CameraServer et CameraFeed

- CameraServer :
 - feeds : Renvoie un tableau de CameraFeed (flux caméra d'entrée)
- CameraFeed :
 - feed_is_active : booléen qui indique si le feed est actif
 - get_id() : renvoie l'ID du feed
 - get_datatype() : renvoie le type d'image que voit le feed (ex : CameraFeed.FEED_RGB)



Extensions MediaPipe dans Godot

GDMP Demo

- C'est le nom du projet dont on s'inspire pour exploiter les données de MediaPipe dans Godot



Exemple d'utilisation

- Après avoir téléchargé les fichiers tasks qui nous intéressent, on peut les utiliser de la sorte :

```
func _setup_mediapipe():  
>I  if not FileAccess.file_exists(model_path):  
>I  >I  print("ERREUR : Modèle .task introuvable !")  
>I  >I  return  
>I  var buffer = FileAccess.get_file_as_bytes(model_path)  
>I  var options = MediaPipeTaskBaseOptions.new()  
>I  options.delegate = delegate  
>I  options.model_asset_buffer = buffer  
>I  task = MediaPipePoseLandmarker.new()  
>I  task.initialize(options, running_mode, num_pose)
```

- On peut ensuite exploiter les données de la sorte dans le *process* :

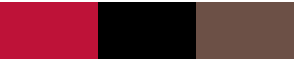
```
var current_fps = Engine.get_frames_per_second()
camera_fps_label.text = "Camera FPS : %d fps" % [current_fps]
```

```
num_label.text = "Marqueurs : 0"
# Récupération de l'image du Viewport
var tex = viewport.get_texture()
var img = tex.get_image()
if not img: return
```

```
# Conversion pour MediaPipe
img.convert(Image.FORMAT_RGBA8)
var mp_image = MediaPipeImage.new()
mp_image.set_image(img)
```

```
# Détection des mains
var start_detect = Time.get_ticks_usec()
var result = task.detect(mp_image)
```

```
if result:
    >I # Dessin des marqueurs sur les mains
    >I var start_render = Time.get_ticks_usec()
    >I var output = renderer.render(mp_image, result.pose_landmarks)
    >I var time_render = (Time.get_ticks_usec()-start_render)/1000.0
    >I #render_label.text = "MediaPipe Render Time : %.2f ms" % [time_render]
    >I
    >I var start_display = Time.get_ticks_usec()
    >I update_debug_overlay(output.image)
    >I var time_display = (Time.get_ticks_usec()-start_display)/1000.0
    >I #display_label.text = "Display Time : %.2f ms" % [time_display]
```



Mains : 1 Right: 0.96 | Open_Palm: 0.63

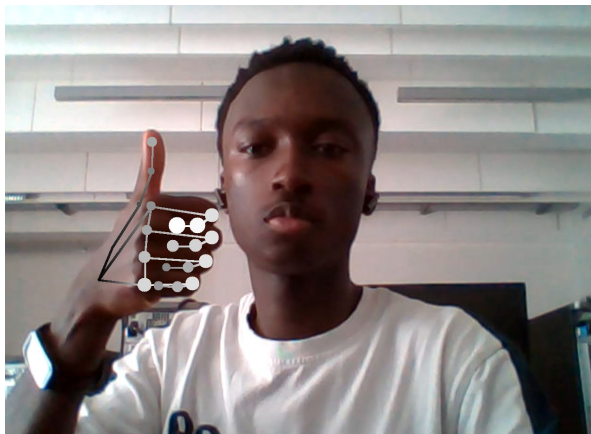
Camera FPS : 12 fps

Time_detect AI : 64.33 ms

MediaPipe Render Time : 14.86 ms

Display Time : 10.39 ms

Coordonnées: x=0.227
y=0.212
z=-0.039
Hand_index : 0



Mains : 1 Right: 0.99 | Thumb_Up: 0.73

Camera FPS : 10 fps

Time_detect AI : 48.94 ms

MediaPipe Render Time : 9.28 ms

Display Time : 8.55 ms

Coordonnées: x=0.192
y=0.494
z=-0.069
Hand_index : 0



Liens utiles

- Google MediaPipe :
<https://ai.google.dev/edge/mediapipe/solutions/guide?hl=fr>
- Godot Docs :
 - Camera Server :
https://docs.godotengine.org/en/stable/classes/class_cameraserver.html
 - Camera Feed :
https://docs.godotengine.org/en/stable/classes/class_camerafeed.html
- CameraServer Extension :
<https://github.com/j20001970/godot-cameraserver-extension>
- GDMP : <https://github.com/j20001970/GDMP>